Offchain Labs Security Council Update

Security Assessment (Summary Report)

September 15, 2025

Prepared for:
Harry Kalodner, Steven Goldfeder, and Ed Felten
Offchain Labs

Prepared by: **Simone Monica and Jaime Iglesias**

Table of Contents

Table of Contents **1**
Project Summary **2**
Project Targets **3**
Executive Summary **4**
Summary of Findings **5**
Detailed Findings **6**
1. rotateNominee allows rotation to a noncompliant nominee 6
2. Cohort replacement involving duplicate members can fail 8
A. Vulnerability Categories **12**
B. Code Quality Recommendations **14**
C. Fix Review Results **16**
Detailed Fix Review Results 17
D. Fix Review Status Categories **18**
About Trail of Bits **19**
Notices and Remarks **20**

Project Summary

Contact Information

The following project manager was associated with this project:

Mary O'Brien, Project Manager
mary.obrien@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Simone Monica, Consultant
simone.monica@trailofbits.com

Jaime Iglesias, Consultant
jaime.iglesias@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
August 29, 2025	Delivery of report draft
August 29, 2025	Report readout meeting
September 3, 2025	Delivery of final summary report
September 4, 2025	Completion of fix review
September 15, 2025	Delivery of final summary report

Project Targets

The engagement involved reviewing and testing the following target.

Governance

Repository	https://github.com/ArbitrumFoundation/governance/
Version	d89b171429e3c4e6c1b7e5ef559a93aef5d35d8a
Type	Solidity
Platform	EVM

Executive Summary

Engagement Overview

Offchain Labs engaged Trail of Bits to review the security of the changes made to the Security Council contracts. These changes include longer cohort durations and adjusted thresholds, as described in the associated governance proposal, to make the election process more effective and practical.

A team of two consultants conducted the review from August 20 to August 29, 2025, for a total of one engineer-week of effort. With full access to source code and documentation, we performed static and dynamic testing of the target, using automated and manual processes.

Observations and Impact

Overall, we found that the changes were easy to reason about and were correctly implemented. We identified only minor issues, one rated as medium severity and one as informational; the medium-severity issue relates to a lack of validation during nominee rotation, which could cause duplicated nominees to be inserted into a cohort and potentially the cohort rotation to fail, if a duplicate nominee goes unnoticed.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that Offchain Labs take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	rotateNominee allows rotation to a noncompliant nominee	Data Validation	Informational
2	Cohort replacement involving duplicate members can fail	Data Validation	Medium

Detailed Findings

1. rotateNominee allows rotation to a noncompliant nominee

Severity: **Informational**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-SC-1
Target: `security-council-mgmt/governors/SecurityCouncilNomineeElectionGovernor.sol`	

Description

The `rotateNominee` function is a new function added to allow a nominee to rotate their address until three days before the compliance phase of the election ends. However, the implementation does not check if the `newNomineeAddress` is a compliant nominee; only the `msg.sender` is checked to be compliant (figure 1.1), allowing a noncompliant nominee to be added back as a nominee.

```
    /// @notice Allows a nominee to rotate their position to a new address
    /// @param proposalId The id of the proposal
    /// @param newNomineeAddress The new address to rotate to
    /// @param signature  A signature from the new member address over the 712
rotateNominee hash
    function rotateNominee(uint256 proposalId, address newNomineeAddress, bytes
calldata signature)
        external
    {
        ElectionInfo storage election = _elections[proposalId];

        if (!isCompliantNominee(proposalId, msg.sender)) {
            revert NotCompliantNominee(msg.sender);
        }
        ...
```

Figure 1.1: Part of the `rotateNominee` function

The severity of this issue is informational, as it does not cause any problems in later steps, such as the voting process, where the nominee the user is voting for is checked to be compliant.

Recommendations

Short term, add an additional check in the `rotateNominee` function to prevent rotations to noncompliant addresses.

Long term, improve the testing suite by adding tests validating that a noncompliant nominee cannot be added as a nominee during rotation.

2. Cohort replacement involving duplicate members can fail

Severity: **Medium**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-SC-2
Target: `SecurityCouncilManager.sol`	

Description

The `rotateNominee` function does not check if the new nominee is already a nominee of the current cohort. This allows the same nominee to appear multiple times in the nominees array. If the duplicated nominee gets enough votes and is already part of the new cohort, the actual replacement transaction will revert due to an attempt to add a member that is already present.

The new cohort will contain the members returned by the `topNominees` function (figure 2.1). Note that the `_compliantNominees` function uses the `nominees` array associated with the `proposalId` election and filters out only noncompliant nominees; hence, it can return the same nominee multiple times, if present. Additionally, there is no deduplication check in the `topNominees` or `selectTopNominess` functions.

```
   function topNominees(uint256 proposalId) public view returns (address[] memory) {
        address[] memory nominees = _compliantNominees(proposalId);
        uint240[] memory weights = new uint240[](nominees.length);
        ElectionInfo storage election = _elections[proposalId];
        for (uint256 i = 0; i < nominees.length; i++) {
            weights[i] = election.weightReceived[nominees[i]];
        }
        return selectTopNominees(nominees, weights, _targetMemberCount());
    }

    /// @notice Gets the top K nominees from a list of nominees and weights.
    /// @param nominees The nominees to select from
    /// @param weights  The weights of the nominees
    /// @param k        The number of nominees to select
    function selectTopNominees(address[] memory nominees, uint240[] memory weights,
uint256 k)
        public
        pure
        returns (address[] memory)
    {
        if (nominees.length != weights.length) {
            revert LengthsDontMatch(nominees.length, weights.length);
        }
        if (nominees.length < k) {
```

```
            revert NotEnoughNominees(nominees.length, k);
        }

        uint256[] memory topNomineesPacked = new uint256[](k);

        for (uint16 i = 0; i < nominees.length; i++) {
            // The nominee's index in the address array is stored in the 16
rightmost bits; the remaining bits store the nominee's weight
            uint256 packed = (uint256(weights[i]) << 16) | i;
            // Packed weight/index values can be compared when comparing weights,
since the values of the weights will outweigh any difference in index;
            // the index value only takes effect here as tie-breaker if the weights
are equal.
            // If the current weight is greater than the smallest of the top-6
weights so far, replace the smallest element with it and re-sort.
            if (topNomineesPacked[0] < packed) {
                topNomineesPacked[0] = packed;
                LibSort.insertionSort(topNomineesPacked);
            }
        }

        address[] memory topNomineesAddresses = new address[](k);
        for (uint16 i = 0; i < k; i++) {
            // retrieve the index from the packed value to look up the nominee's
address.
            topNomineesAddresses[i] = nominees[uint16(topNomineesPacked[i])];
        }

        return topNomineesAddresses;
    }
```

Figure 2.1: The `topNominees` and `selectTopNominees` functions

The `SecurityCouncilMemberElectionGovernor` contract's `_execute` function calls the `replaceCohort` function with the `topNominees` as members of the new cohort (figure 2.2).

```
   function _execute(
        uint256 proposalId,
        address[] memory, /* targets */
        uint256[] memory, /* values */
        bytes[] memory callDatas,
        bytes32 /* descriptionHash */
    ) internal override {
        // we know that the election index is part of the calldatas
        uint256 electionIndex = extractElectionIndex(callDatas);

        // it's possible for this call to fail because of checks in the security
council manager
        // getting into a state inconsistent with the elections, if it does the
Security Council
        // will need to update the Manager so that this replaceCohort can go through
```

```
        // Otherwise this and future elections will remain blocked.
        securityCouncilManager.replaceCohort({
            _newCohort: topNominees(proposalId),
            _cohort: electionIndexToCohort(electionIndex)
        });
    }
```

Figure 2.2: The `_execute` function in `SecurityCouncilMemberElectionGovernor`

Finally, the `replaceCohort` function deletes the old cohort and adds the new members to it. When adding the members, it checks if the new member is present either in the first or second cohort and reverts if so (figure 2.3).

```
   function replaceCohort(address[] memory _newCohort, Cohort _cohort)
        external
        onlyRole(COHORT_REPLACER_ROLE)
    {
        ...
        // delete the old cohort
        _cohort == Cohort.FIRST ? delete firstCohort : delete secondCohort;

        for (uint256 i = 0; i < _newCohort.length; i++) {
            _addMemberToCohortArray(_newCohort[i], _cohort);
        }
        ...
    }

   function _addMemberToCohortArray(address _newMember, Cohort _cohort) internal {
        ...
        address[] storage cohort = _cohort == Cohort.FIRST ? firstCohort :
secondCohort;
        if (cohort.length == cohortSize) {
            revert CohortFull({cohort: _cohort});
        }
        if (firstCohortIncludes(_newMember)) {
            revert MemberInCohort({member: _newMember, cohort: Cohort.FIRST});
        }
        if (secondCohortIncludes(_newMember)) {
            revert MemberInCohort({member: _newMember, cohort: Cohort.SECOND});
        }

        cohort.push(_newMember);
        ...
```

Figure 2.3: The `replaceCohort` and `_addMemberToCohortArray` functions

While less likely, the duplication scenario can also happen in the `addContender` function if the proposed contender is part of the current cohort, since it is automatically added as a nominee without checking if the address is already a nominee. This scenario could happen if the `rotateNominee` function is used to rotate a nominee to that address, and after that, the `addContender` function is called.

```
   function addContender(uint256 proposalId, bytes calldata signature) external {
        ...
        // if the signer is part of the outgoing cohort, we automatically add them
as a nominee
        if (securityCouncilManager.cohortIncludes(currentCohort(), signer)) {
            _addNominee(proposalId, signer); // @note duplicate nominee if
rotateNominee is called before this with the same signer address
        }
    }
```

Figure 2.4: The `addContender` function

It is important to note that the `nomineeVetter` would have three days to exclude the duplicated nominee and avoid this issue completely.

Exploit Scenario

Alice calls the `rotateNominee` function with the address of an existing cohort nominee. As a result, the address is included twice in the nominee list. This goes unnoticed and leads to a failed cohort rotation.

Recommendations

Short term, include additional checks in the `rotateNominee` function to prevent existing nominees (from either cohort) from being used for rotation.

Long term, thoroughly document the necessary checks that need to be implemented during nominee rotation and use them to drive testing further (e.g., by including tests in which existing nominees are used for rotation).

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Recommendations

The following recommendations are not associated with any specific vulnerabilities. However, they will enhance code readability and may prevent the introduction of vulnerabilities in the future.

- **The following comment is awkwardly phrased and may contain typos, which could cause confusion.** Revise the comment for clarity.

```
// check to make sure the new nominee is not part of the other cohort (the cohort
not currently up for election)
// this only checks against the current the current other cohort, and against the
current cohort membership
// in the security council, so changes to those will mean this check will be
inconsistent.
// this check then is only a relevant check when the elections are running as
expected - one at a time,
// every 6 months. Updates to the sec council manager using methods other than
replaceCohort can effect this check
// and it's expected that the entity making those updates understands this.
if (securityCouncilManager.cohortIncludes(otherCohort(), newNomineeAddress)) {
    revert AccountInOtherCohort(otherCohort(), newNomineeAddress);
}
```

Figure B.1: Part of the `addContender` function

- **The following comment should state "every `cadenceInMonths`" rather than "every 6 months."**

```
// check to make sure the new nominee is not part of the other cohort (the cohort
not currently up for election)
// this only checks against the current the current other cohort, and against the
current cohort membership
// in the security council, so changes to those will mean this check will be
inconsistent.
// this check then is only a relevant check when the elections are running as
expected - one at a time,
// every 6 months. Updates to the sec council manager using methods other than
replaceCohort can effect this check
// and it's expected that the entity making those updates understands this.
if (securityCouncilManager.cohortIncludes(otherCohort(), newNomineeAddress)) {
    revert AccountInOtherCohort(otherCohort(), newNomineeAddress);
}
```

Figure B.2: Part of `rotateNominee` function

- **The following comment should specify that "this is 0.1% of votable tokens," not "0.2%."**

```
/// @notice Gets the top K nominees with greatest weight for a given proposal,
///         where K is the manager.cohortSize()
/// @dev    Care must be taken of gas usage in this function.
///         This is an O(n) operation on all compliant nominees in the nominees
governor.
///         The maximum number of nominees is set by the threshold of votes required
to become a nominee.
///         Currently this is 0.2% of votable tokens, which corresponds to 500 max
nominees.
///         Absolute worst case, this function uses 4502345 with 500 nominees, or
about 9k gas per nominee (when called externally).
/// @param proposalId The proposal to find the top nominees for
function topNominees(uint256 proposalId) public view returns (address[] memory) {
```

Figure B.3: Part of the `topNominees` function

C. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On September 4, 2025, Trail of Bits reviewed the fixes and mitigations implemented by the Offchain Labs team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, Offchain Labs has resolved the two issues disclosed in this report. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	rotateNominee allows rotation to a noncompliant nominee	Informational	Resolved
2	Cohort replacement involving duplicate members can fail	Medium	Resolved

Detailed Fix Review Results

TOB-SC-1: rotateNominee allows rotation to a noncompliant nominee
Resolved in PR #361. The function now checks whether the new nominee address is excluded.

TOB-SC-2: Cohort replacement involving duplicate members can fail
Resolved in PR #361. The `_addNominee` helper function now checks whether the address is already a nominee, thus preventing the issue.

D. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.